

腾讯安全 | 云鼎实验室

锋刃无影 御见未来

腾讯安全沙龙第3期（成都站）



IFS

关于我

About me

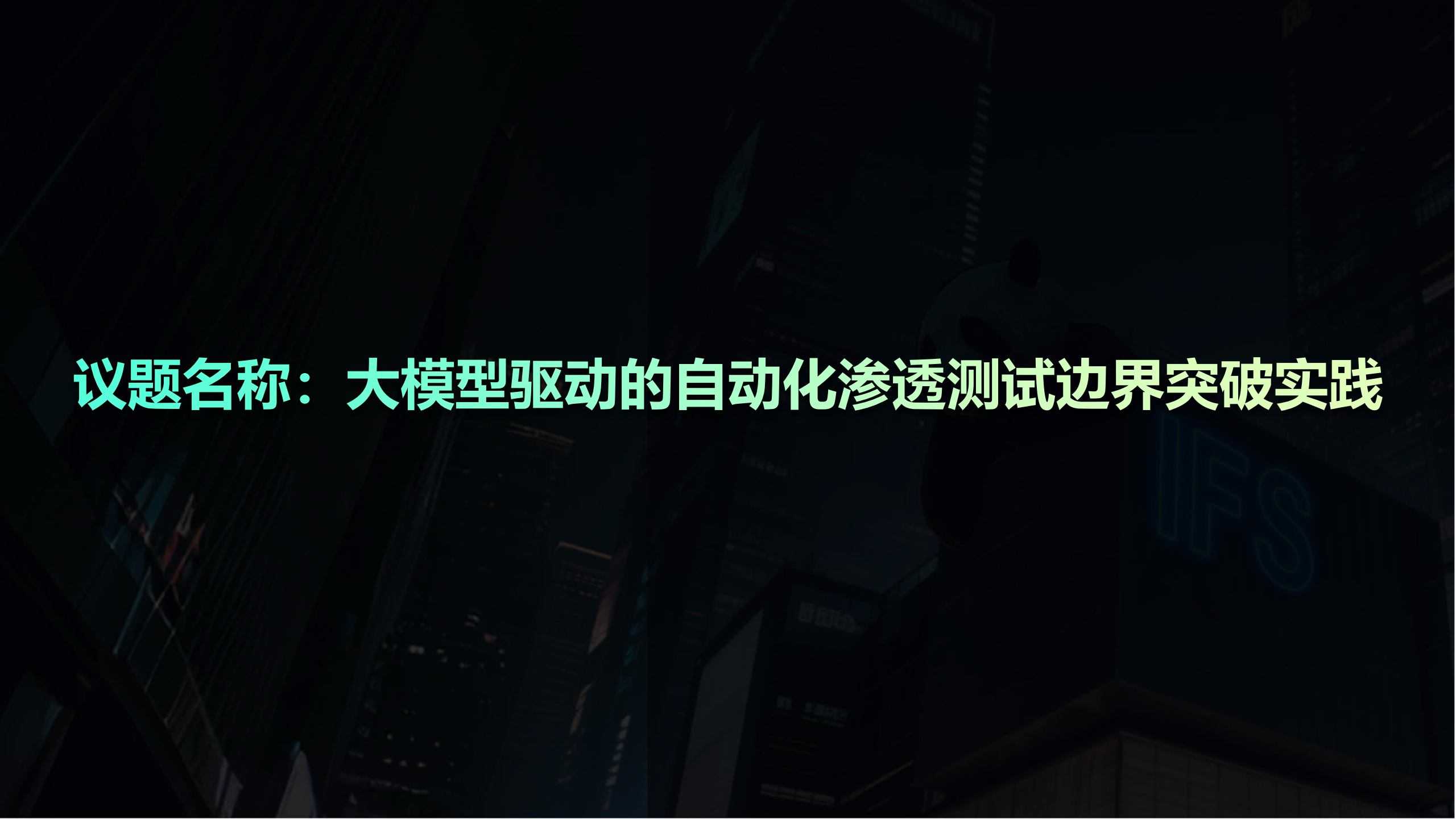
刘浩

绿盟科技高级研究员

聚焦“AI渗透”与“大模型PoC生成”等研究与实践

自动化渗透测试工具——EZ核心开发人员

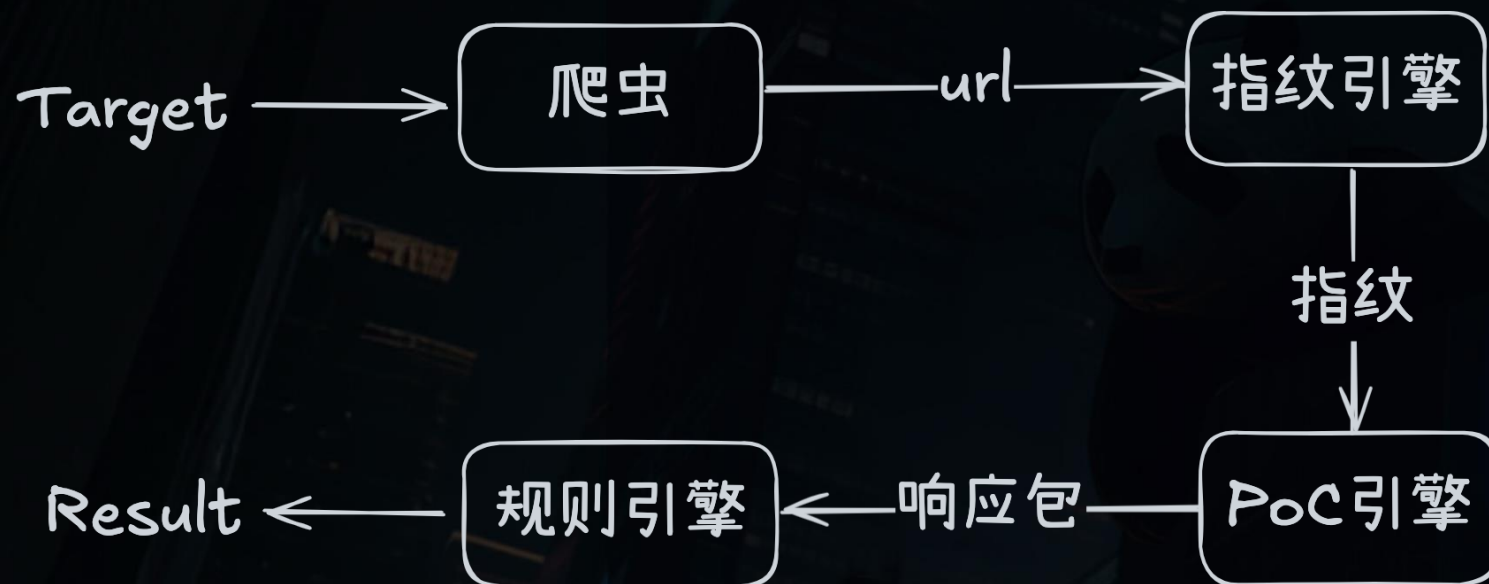
M-SEC社区核心共创成员



议题名称：大模型驱动的自动化渗透测试边界突破实践

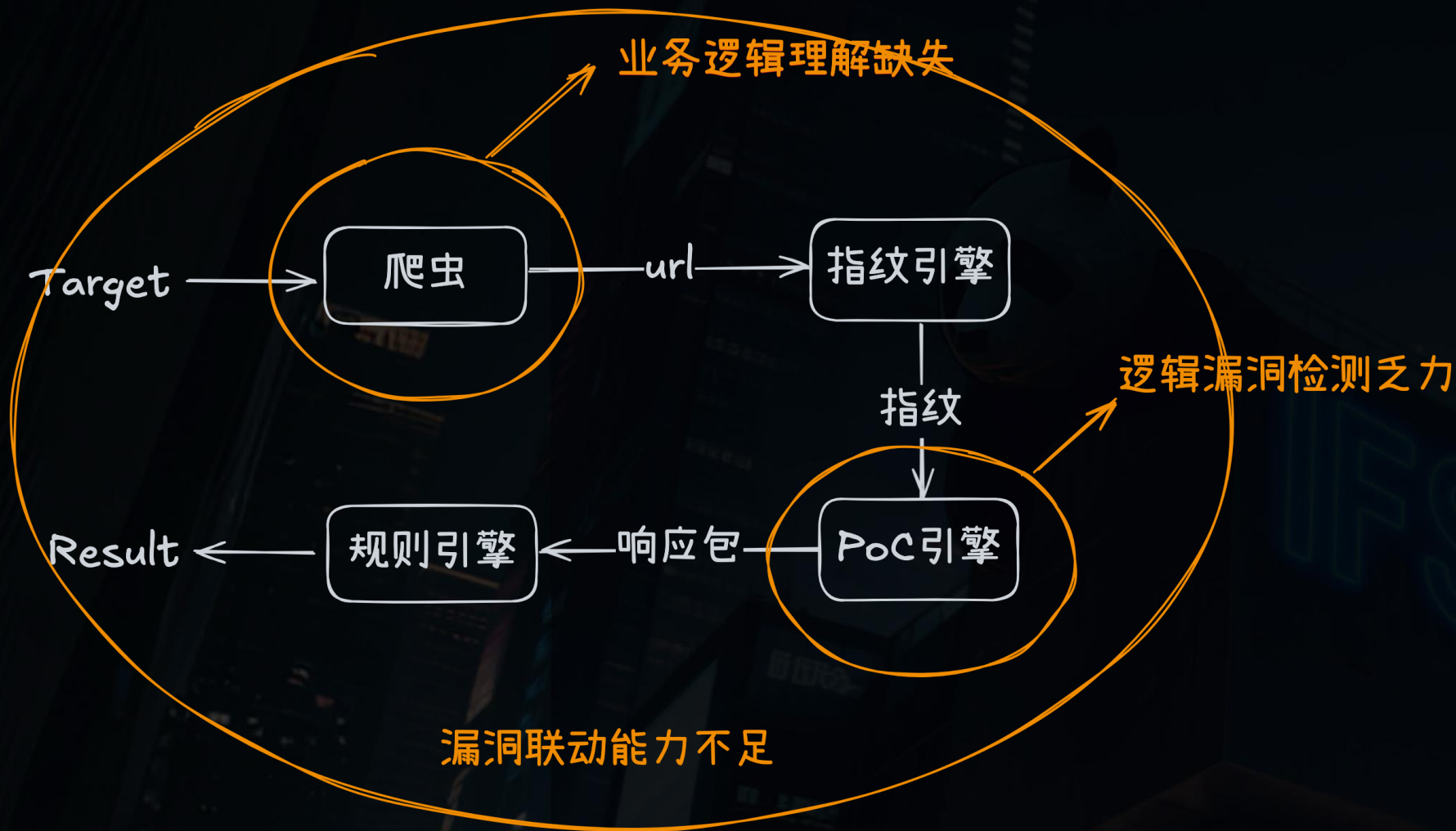
1、传统之困：自动化渗透测试工具的瓶颈

自动化渗透测试工具的瓶颈



自动化渗透测试工具的工作原理

自动化渗透测试工具的瓶颈



2、初探 LLM：期望与现实

TEST1: ALL IN LLM !

LLM 对于基础漏洞检出能力不太理想，因为基于黑盒的 LLM 的渗透测试需要经历 Payload 生成、网站交互、响应包理解等一系列过程，一个功能测试就需要很长时间，**效率与漏洞产出比太低**。



```
[{"payload": "lmsrcbuop'><svg/onload=alert(1);>", "response": ["<h2  
align=\"center\">&#27809;&#26377;&#25214;&#21040;&#21644;l  
mrescbuop'&gt;&lt;svg/onload=alert(1) &gt;&#30456;&#20851;&#3  
0340;&#32467;&#26524;.</h2>"]}]]
```

json

Copy code

```
{  
  "done": false,  
  "payload": "lmsrcbuop'><iframe/src=javascript:alert(1);>",  
  "message": "HTML entities are still being encoded. Testing an iframe-based payload"  
}
```


TEST1: ALL IN LLM !

在自动

题，而

ChatGPT o1-preview ▾

9.11 or 9.8 - which is bigger?



Thought for 5 seconds ^

COMPARING DECIMALS

Analyzing unit magnitude

I'm working through the difference between 9.8 m/s^2 , the acceleration due to gravity, and 9.11. The latter numerically exceeds the former, though its context remains vague.

9.11 is larger than 9.8.



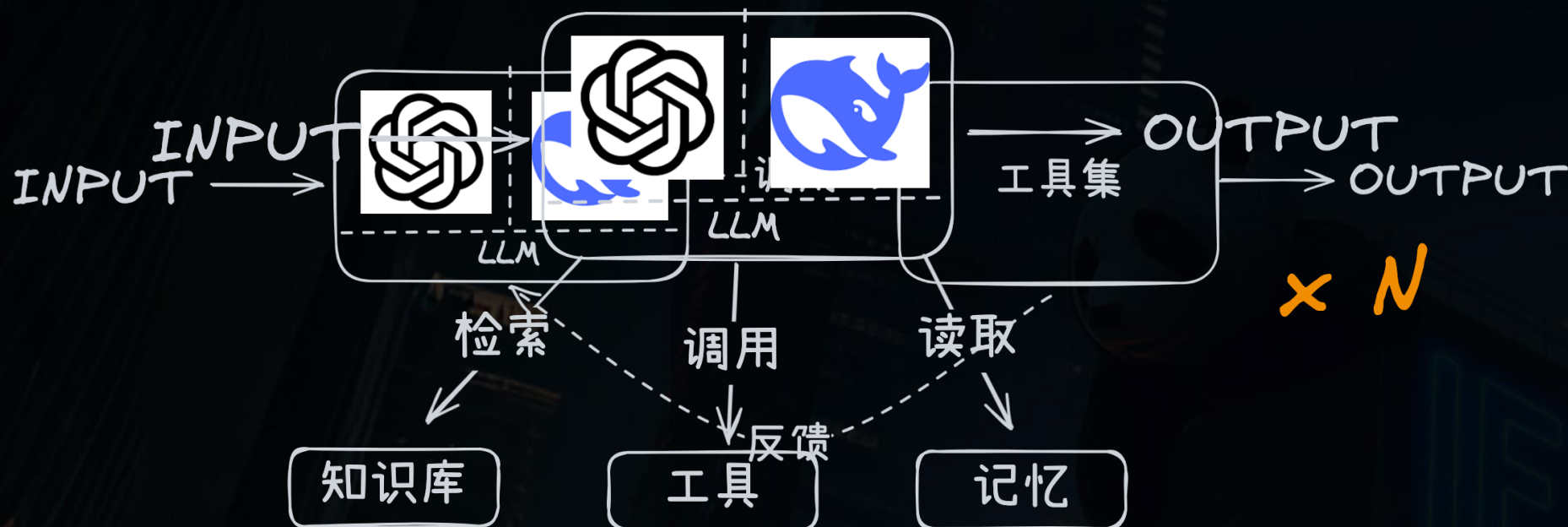
TEST2: 传统工具 + Workflow

对特定场景通过预定义的代码路径对 LLM 和工具进行编排，但没有自主规划，流程较固定，适合规则和流程明确的场景。



对渗透测试报告中的 SQL 注入漏洞进行复测，要拿到数据库名。

TEST2: 传统工具 + Workflow



LLMs 和工具通过预定义的代码路径进行编排

LLMs 动态地指导自己的流程和工具使用，保持对完成任务方式的控制

TEST3: 传统工具 + Agent

Agent：只要定好初始任务和工具，大模型会根据工具的反馈，决定下一步调用的工具和相应的动作，直到任务完成，适合开放性的场景。



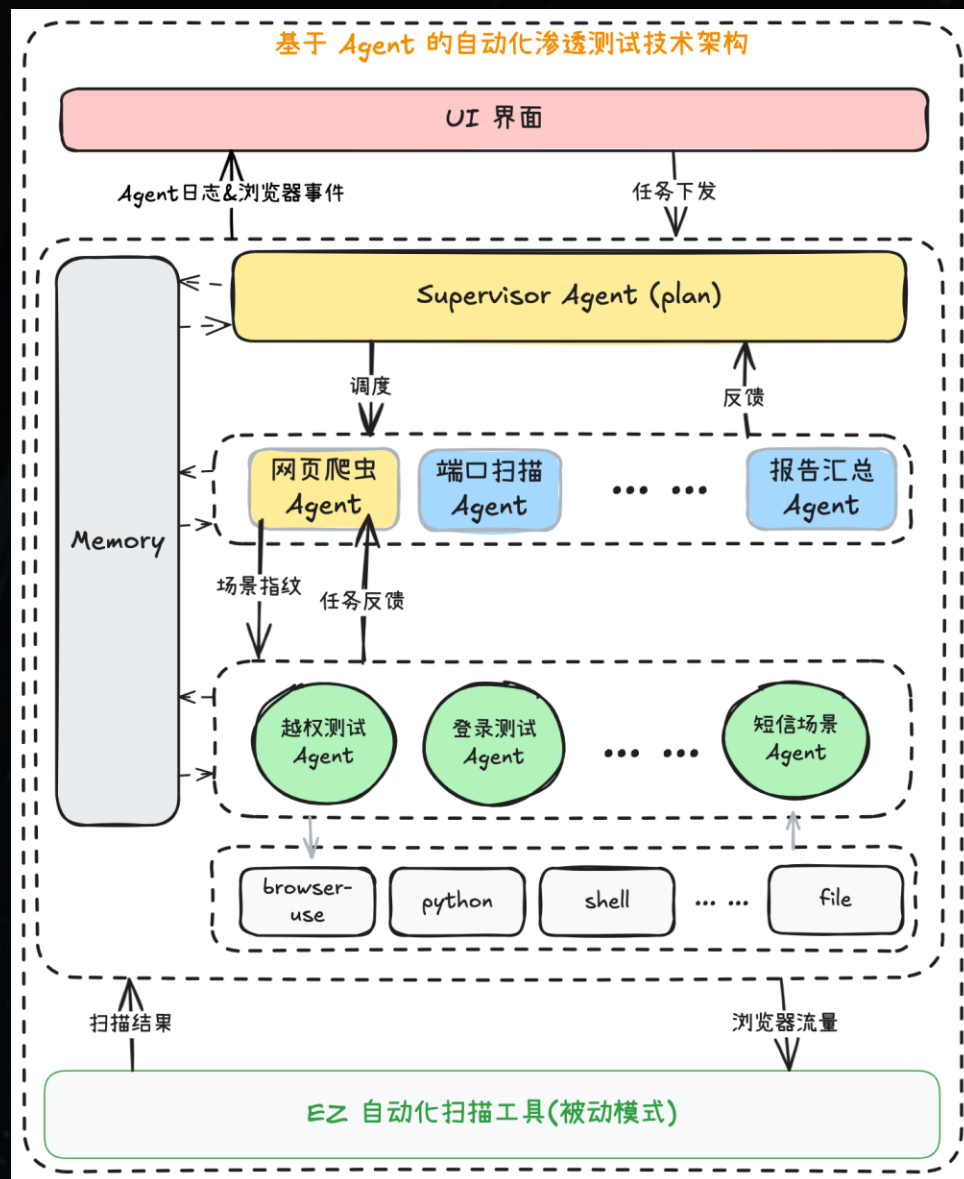
Agent or Workflow

特征维度	Workflow (预定义流程)	Agent (自主智能体)
核心驱动	预定义的规则和代码编排	大模型的自主规划与推理能力
任务类型	结构化、流程明确的重复性任务	开放性、动态变化的复杂任务
决策机制	基于 固定规则 和 顺序执行	基于 环境感知 和 目标导向
主要优势	稳定、高效、可控	适应性强、能处理复杂和未知问题
潜在挑战	难以扩展到新场景	结果不稳定、对 LLM 依赖高

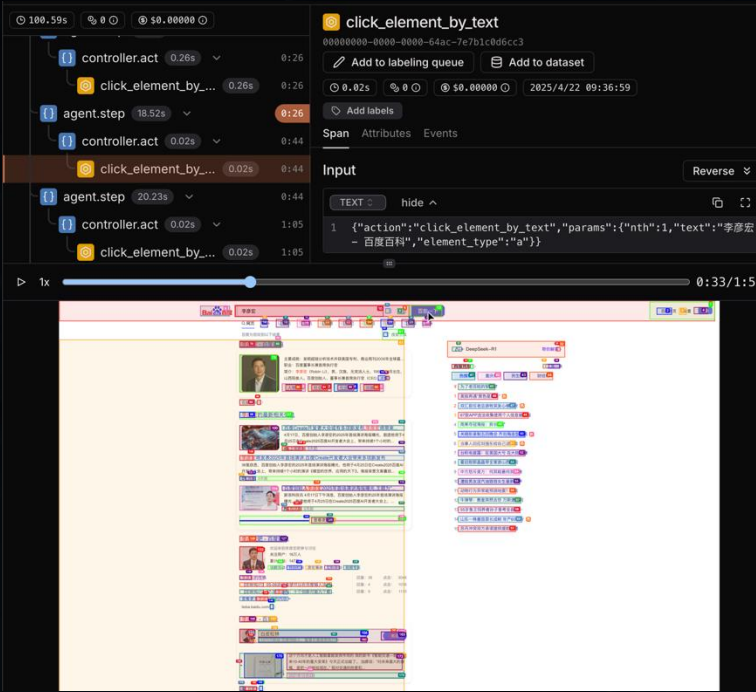
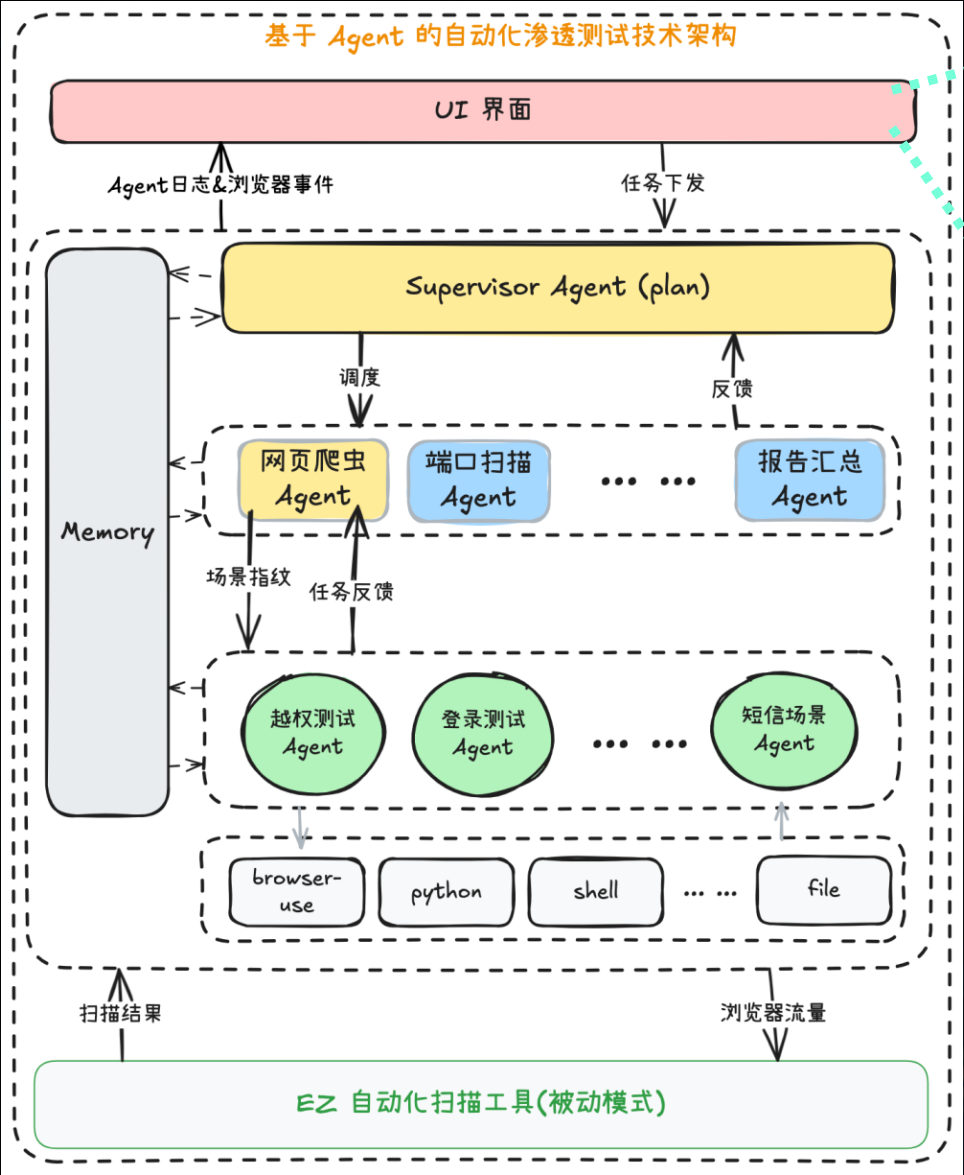
使用 Agent 进行宏观规划和探索，调用具体的 Workflow 来高效、稳定地执行其中的确定性步骤。

3、实践框架：构建 Agent 驱动的渗透测试体系

基于 Agent 的自动化渗透测试框架



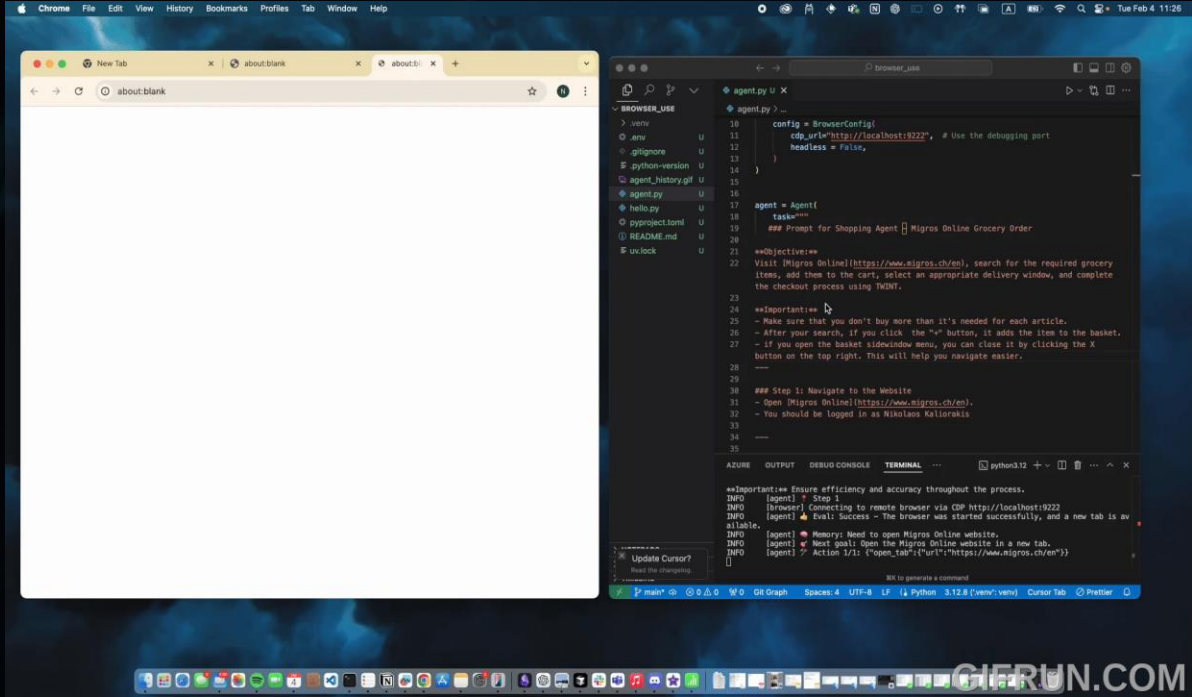
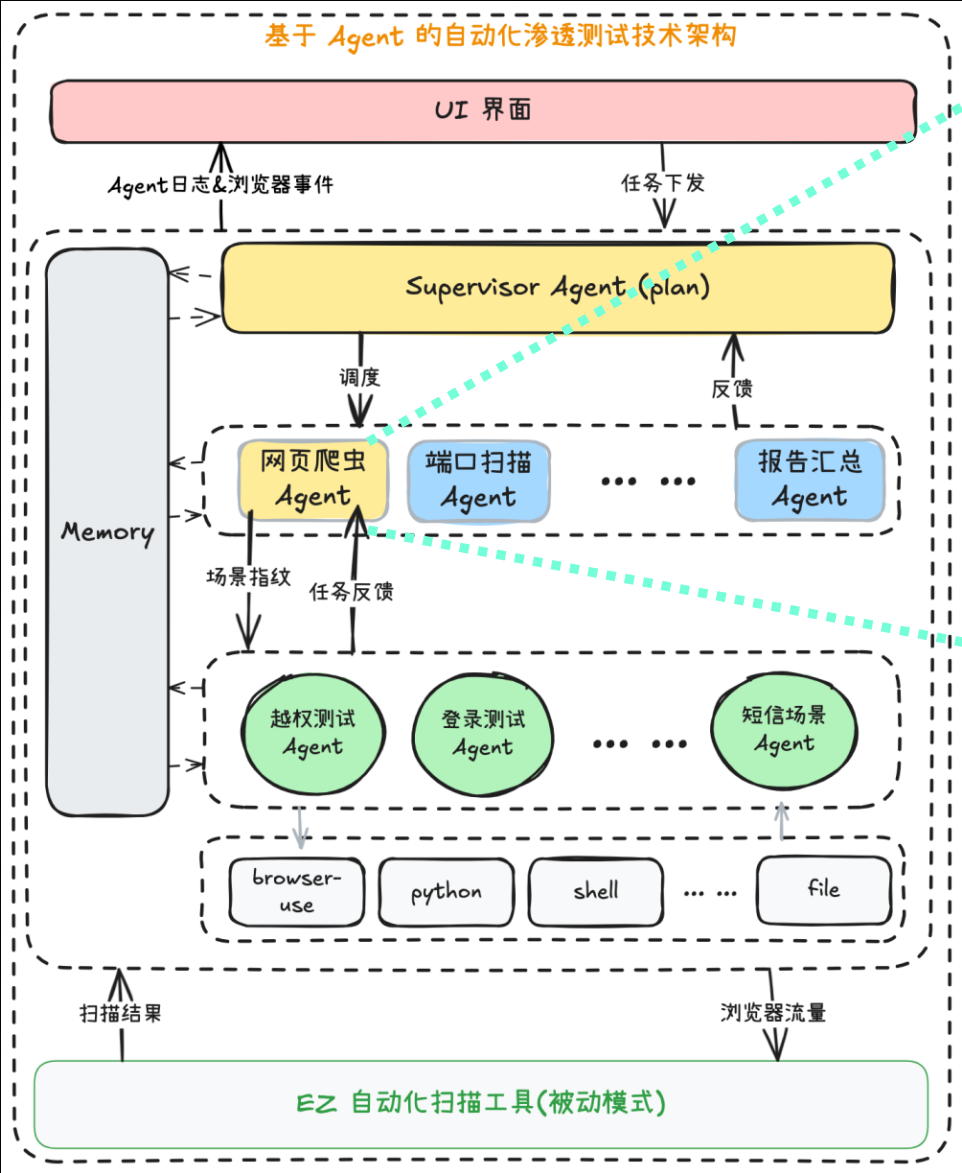
基于 Agent 的自动化渗透测试框架



任务下发后，前端界面会实时响应 Agent 的活动、工具调用过程和浏览器画面。

借助开源 AI 应用日志追踪平台 Laminar，支持渗透过程的可视化回溯，能够完整记录测试的每一项功能和每一个测试步骤。

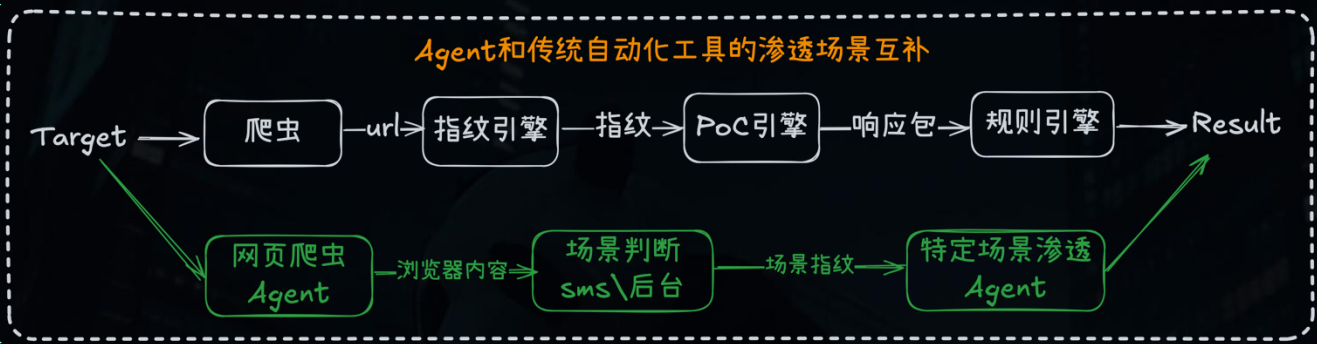
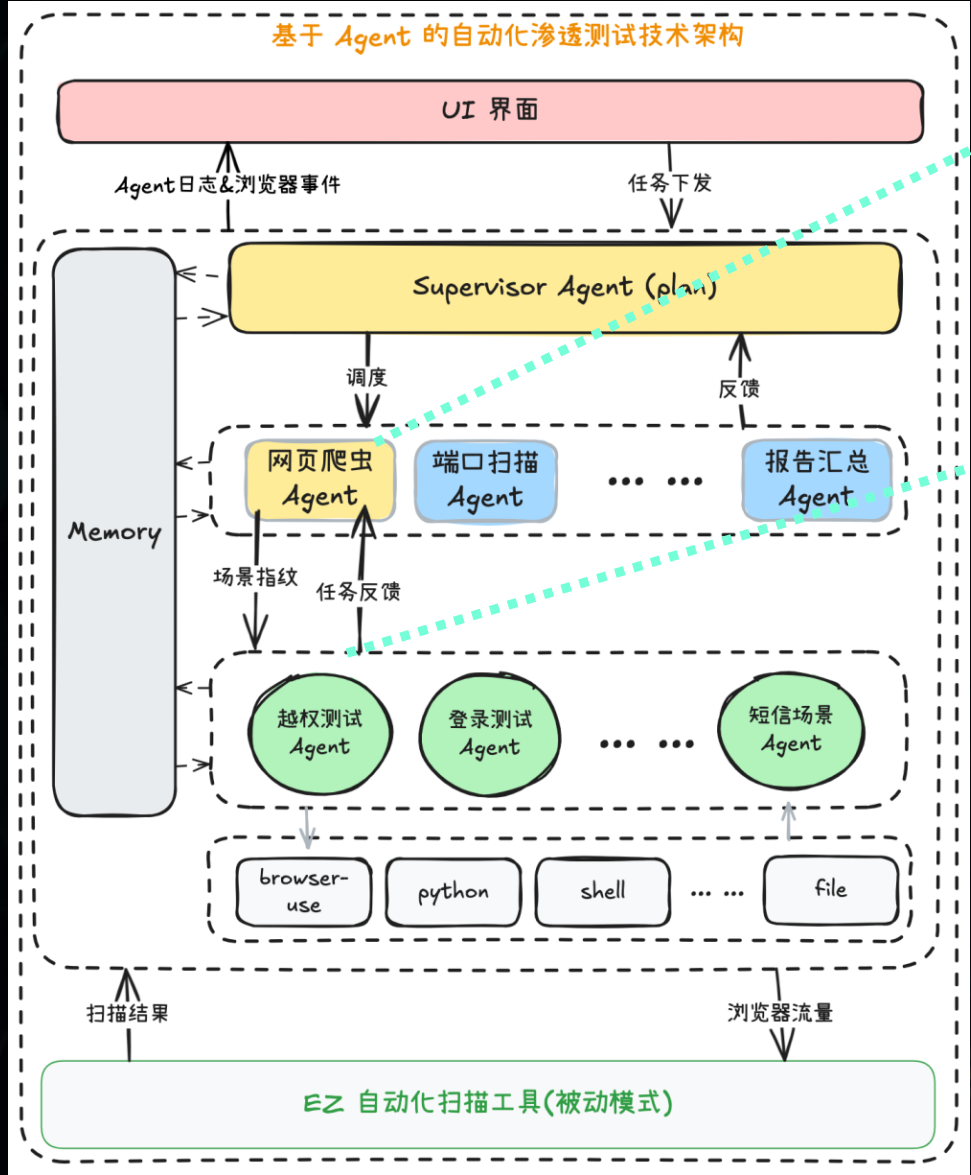
基于 Agent 的自动化渗透测试框架



网页爬虫 Agent 可以模拟人操作和理解业务：基于视觉模型识别页面元素，点击按钮、填写表单、跳转页面等。

将业务流量导入自动化渗透测试工具，缓解传统爬虫无法理解业务逻辑和扫描工具覆盖率低的问题。

基于 Agent 的自动化渗透测试框架



针对传统自动化渗透测试无法覆盖的场景，可以将最佳的渗透测试实践固化为特定场景的渗透测试 Agent (Agent PoC)。

通过大模型的视觉理解能力，对页面进行场景判断，根据“场景指纹”调度“特定场景渗透 Agent”进行逻辑漏洞的突破。

4、深度剖析：特定场景渗透测试 Agent 设计实例

短信验证码/登录场景渗透测试 Agent 设计

欢迎登录~

+86 请输入手机号

请输入图形验证码



请输入短信验证码

获取验证码

登录

短信轰炸

登录绕过

图形验证码绕过

短信验证码爆破

短信验证码/登录场景渗透测试 Agent 设计

欢迎登录~

+86 请输入手机号

请输入图形验证码



请输入短信验证码

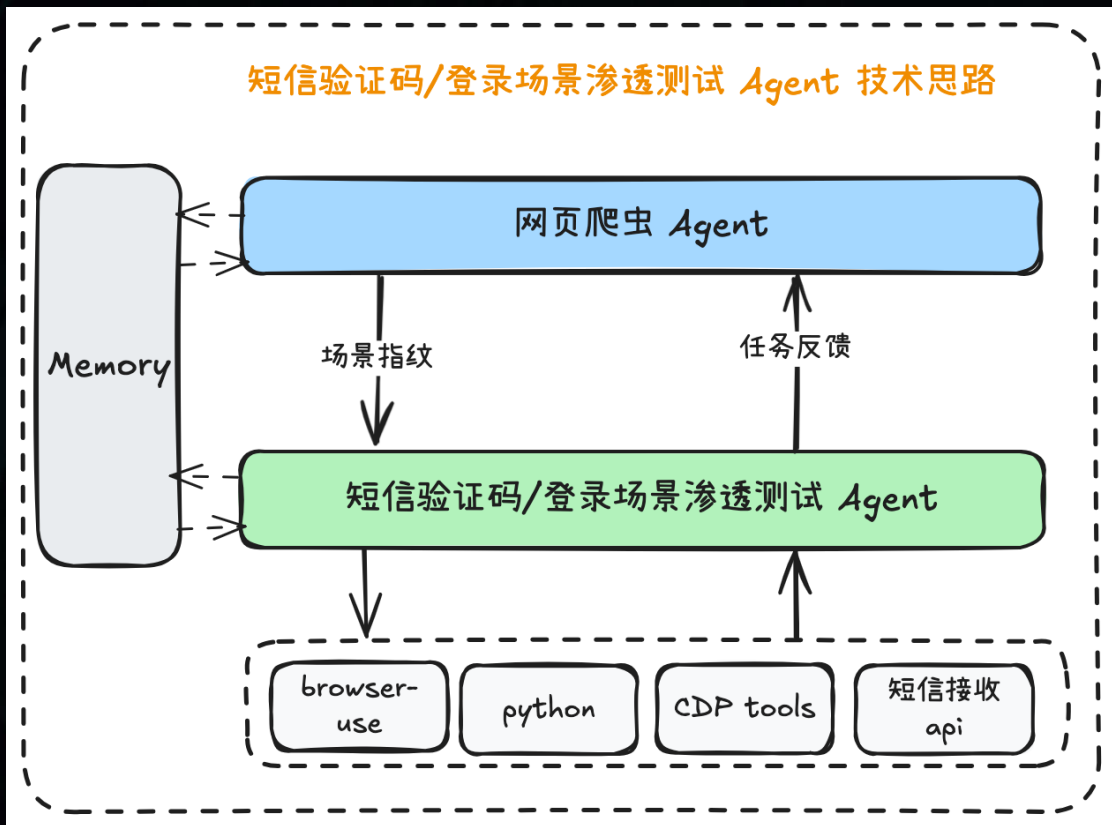
获取验证码

登录

```
POST /Login HTTP/1.1
Host :
Connection: keep-alive
Content-Length auto: 100
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
sec-ch-ua: "Google Chrome";v="135", "Not-A.Brand";v="8", "Chromium";v="135"
Content-Type: application/json; charset=UTF-8
sec-ch-ua-mobile: ?0
Referer:
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: zh-CN, zh; q=0.9

{"phone": "Pekkq+fuxTXsx36dDV4hVQ==", "code": "aFILDcx9zFQfvUNsDm2N1A==",
"commonSign": null, "time": null}
```

短信验证码/登录场景渗透测试 Agent 设计



1. 分析 JS 代码得到的加密参数

原始 JS 代码如下：

```

var r = 3452; n = o.n(r); const i = ["B", "6", "4", "E", "B", "5", "8", "6", "8", "9", "D", "7", "A", "E", "A", "8", "D", "2", "4", "7", "1", "F", "B", "2", "2", "6", "G", "D", "F", "D", "D", "2", "6", "4", "A", "B", "D", "E", "7", "1", "1", "F", "B", "2", "6", "G", "D", "F", "F", "C"];
c = i.reverse().join, a = ["enc", "toString", "substring", "pad", "parse", "decrypt", "encrypt", "Utf8", "Base64", "mode" instanceof Stringify], s = function(t, e) {
  t = 0; let n = a[t]; return n; } function ut(t) { const e = o.a["enc"]["Utf8"], n = o.a["enc"]["Utf8"], p = o.a["enc"]["Base64"], m = o.a["enc"]["mode"]; s = function(t, e) {
    o.a[s["0x06"]]["Utf8"]s["0x04"], i = o.a["AES"]s["0x06"]}(r, e, { iv: n, mode: o.a[s["0x09"]]["CBC"], padding: o.a["pad"]["Pkcs7"]}); return
    o.a["enc"]s["0x08"]s["0x04"]}); }

```

分析过程说明：

```

290     model = EmbeddingEmbedderWrapper(model=generator, task=task, up_down=generator.get_up_down,
291     agent = Agent(
292         task=task,
293         llm=model,
294         controller=controller,
295         browser=browser, # 传入 browser 实例
296         context=context # 确保 Agent 使用了带拦截器的上下文
297     )
298
299     await agent.run() # 运行 Agent
300     [?] [?] ↑ Delete: await self.continue_request(interception_id)
301

```

问题 输出 调试控制台 终端 端口 GITLENS AUGMENT NEXT EDIT 1

```
INFO [controller] selcted option 2 with value ['2'] in frame 0
--- DEBUG: handle_intercepted_request called! InterceptionId: interception-job-52.0, Stage: Request ---

--- 拦截到请求: POST http://39.106.145.0:82/vul/sqli/sqli_id.php ---
原始 POST Body: id=2&submit=E6%9FA%E8%AF%A2
尝试修改 POST Body 为: id=5&submit=E6%9FA%E8%AF%A2
设置 postData (未编码): id=5&submit=E6%9FA%E8%AF%A2...
请求 interception-job-52.0 已修改并继续。
--- DEBUG: handle_intercepted_request called! InterceptionId: interception-job-52.0, Stage: Response ---

--- 拦截到响应: POST http://39.106.145.0:82/vul/sqli/sqli_id.php (状态码: 200) ---
检测到 POST 请求到 .php 页面: http://39.106.145.0:82/vul/sqli/sqli_id.php, 准备修改响应体。
注入修改后的响应: modifybycdp...
修改后的响应已发送。

INFO [controller] Clicked button with index 91:
INFO [agent] Step 3
INFO [agent] 🍀 Eval: Success - I have submitted the form with id 2
INFO [agent] 🧠 Memory: I have submitted the form with id 2
INFO [agent] 🎯 Next goal: The task is completed
INFO [agent] 🏁 Action 1.1: {"done":true,"text":"I have submitted the form with id 2","success":true}
```

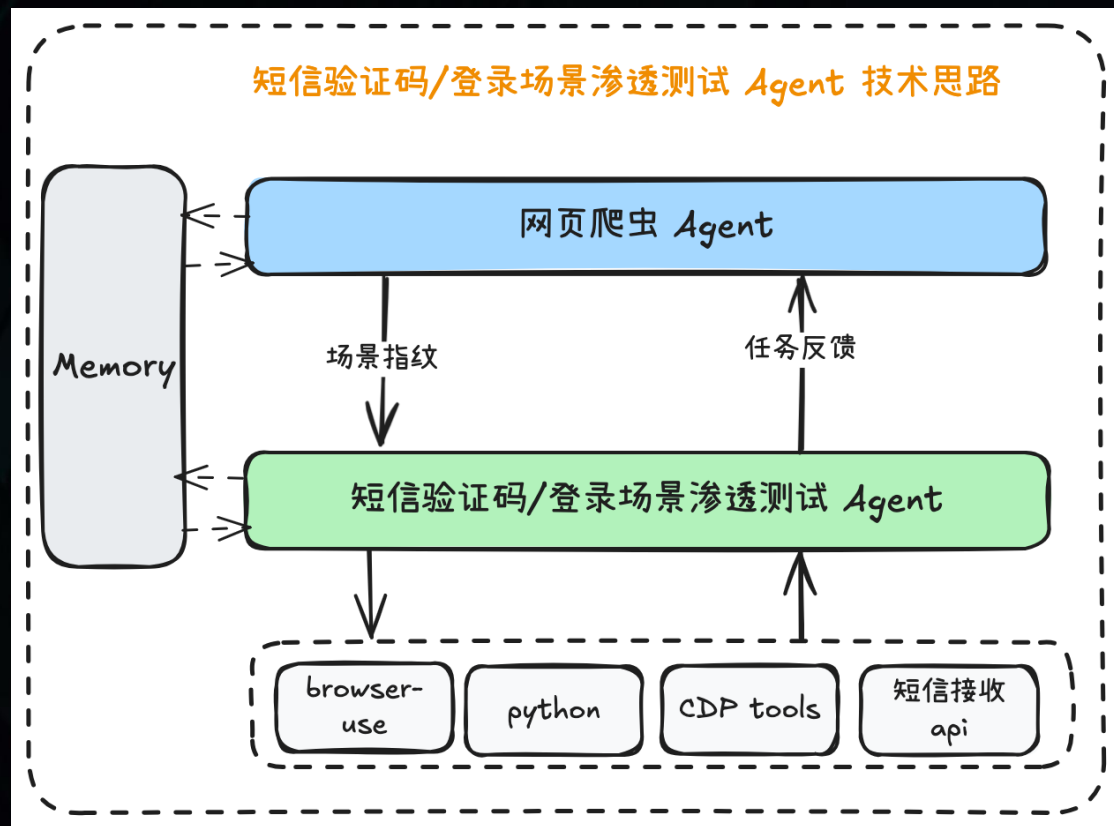
```
data = '13368253742'
data_bytes = data.encode('utf-8')

# 对数据进行 PKCS7 填充 (块大小为 16 字节)
padded_data = pad(data_bytes, AES.block_size)

# 创建 AES CBC 模式加密器
cipher = AES.new(key, AES.MODE_CBC, iv)
encrypted_bytes = cipher.encrypt(padded_data)

# 对加密数据进行 Base64 编码后输出结果
encrypted_base64 = base64.b64encode(encrypted_bytes).decode('utf-8')
print(encrypted_base64)
```

短信验证码/登录场景渗透测试 Agent 设计

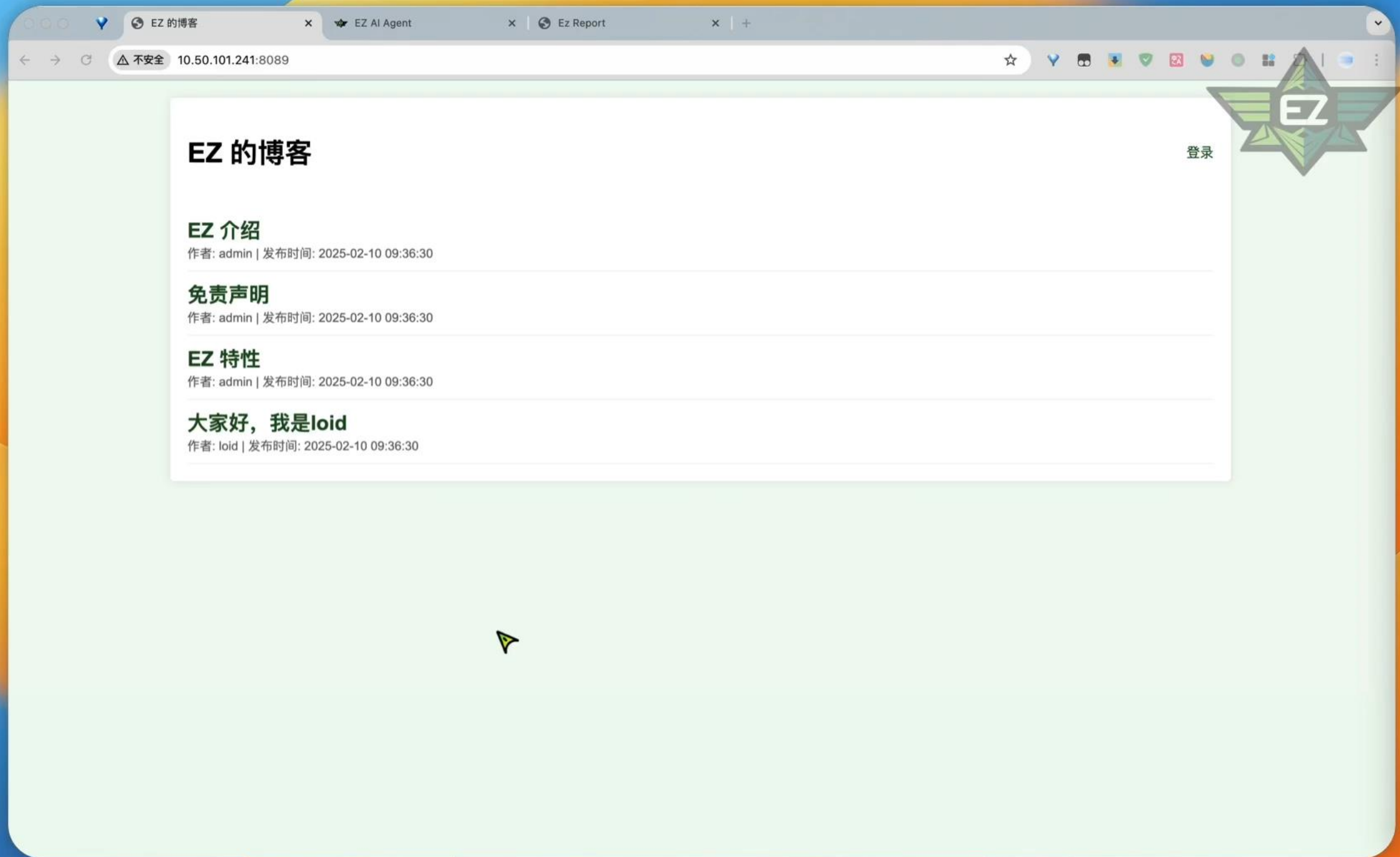


1. 交互与捕捉
-> 填表/点击 (browser-use) -> 捕获 JS (CDP)
2. 分析与准备
-> 分析 JS (LLM) -> 生成加密脚本 (Python)
3. 拦截与攻击
-> 拦截请求 (CDP) -> 加密 Payload (Python)
4. 执行与判定
-> 发送修改请求 (CDP) -> 分析结果 (LLM) -> 漏洞

总结与展望

大模型的发展对自动化渗透测试的影响：

- **核心突破：理解复杂性与动态适应。**
- **效率跃升：实现 7x24 小时不间断、更广业务覆盖的自动化扫描。**
- **流程优化：固化最佳渗透测试实践，提升整体渗透测试的质量和一致性。**
- **能力拓展：有效提升对复杂业务场景、逻辑漏洞的自动化检测能力。**



EZ 的博客

登录

EZ 介绍

作者: admin | 发布时间: 2025-02-10 09:36:30

免责声明

作者: admin | 发布时间: 2025-02-10 09:36:30

EZ 特性

作者: admin | 发布时间: 2025-02-10 09:36:30

大家好，我是loid

作者: loid | 发布时间: 2025-02-10 09:36:30

谢谢



IFS